

# Exploring Object-Oriented Principles in Deep Learning Frameworks: PyTorch vs. TensorFlow

Yutong Zhang, Xujia Zhang

## Abstract

This project analyzes how object-oriented principles appear in two major deep learning frameworks, PyTorch and TensorFlow/Keras. It compares their architectural choices through examples of CNN, optimization, and dataset design. The study finds that PyTorch emphasizes explicit composition and user control through an imperative style, while TensorFlow favors encapsulation and automation via declarative APIs. By mapping these frameworks to classical design patterns—such as Template Method, Strategy, and Facade—the work shows how object-oriented abstraction supports modularity, flexibility, and reusability in machine learning systems. All code and writeup is stored in this repo [https://github.com/XUJIAZHANG2002/CSCI5448\\_project](https://github.com/XUJIAZHANG2002/CSCI5448_project)

## Introduction

PyTorch (Paszke et al. 2019) and TensorFlow (Abadi et al. 2016) are two of the most widely used deep learning frameworks today (Wikipedia Pytorch 2025)(Wikipedia Tensorflow 2025). Both provide high-level tools for building, training, and deploying neural networks, but they differ in how they are designed and how developers interact with them.

This project explores how these frameworks apply object-oriented (OO) principles in their architecture and programming interface. In particular, we examine how core OO concepts, such as encapsulation, inheritance, abstraction, polymorphism, and composition, appear in each system. We focus on the design of `nn.Module` in PyTorch (PyTorch official documents 2024) and `tf.keras.Model` in TensorFlow (Keras official documents 2023), which are the main abstractions for defining neural network models.

By analyzing these frameworks, we aim to understand how OO principles and design patterns support flexibility, modularity, and code reuse in machine learning software. Comparing PyTorch and TensorFlow from this perspective helps us to see how each framework structures its internal design and how users benefit from these abstractions when developing machine learning applications.

## Background: Object-Oriented Design Patterns In Machine Learning

Object-oriented design patterns provide reusable solutions for recurring architectural problems and have become increasingly relevant in modern machine learning systems. Classical foundations such as the "Gang of Four" catalog formalized the core behavioral, structural, and creational patterns that remain applicable across domains (Gamma et al. 1994). Recent works extend these principles to machine learning and deep learning pipelines, emphasizing maintainability, modularity, and human-in-the-loop system integration (Andersen and Maalej 2023)(Chen et al. 2024). Industrial design guides such as *Machine Learning Design Patterns* highlight their practical use for model lifecycle management and reproducibility (Brown, Suryadinata, and others 2023). Together, these works illustrate how traditional object-oriented abstraction and composition principles continue to inform the engineering of large-scale deep learning frameworks.

## Comparing PyTorch & TensorFlow

In this section, we examine two implementations of a convolutional neural network (CNN), a classic deep learning architecture widely used in computer vision. Both versions share the same structure of convolution, pooling, and fully connected layers but differ in their programming paradigms: the PyTorch Model adopts an *imperative* style where computation follows PyTorch control flow, while the TensorFlow Model uses a more *declarative* design built from predefined layer objects and executed within the Keras training loop. We also include an example of loading data in both frameworks, PyTorch Dataset and TensorFlow Dataset, to illustrate how each framework prepares the dataset for the neural network training. These comparisons highlight the object-oriented principles shared by both frameworks, encapsulation of layers and modular composition, as well as their key design differences.

## Model Design

**Layer Composition** Both frameworks rely on classical inheritance and polymorphism to define model hierarchies and enable method overriding. In PyTorch, all layers inherit from `nn.Module`, forming a transparent inheritance chain

(e.g., `Conv2d`  $\rightarrow$  `_ConvNd`  $\rightarrow$  `Module`) that unifies parameter registration, device management, and the polymorphic `forward()` interface. Each subclass customizes its behavior by overriding this method, allowing dynamic and Pythonic execution.

TensorFlow follows a similar structure, where layers such as `tf.keras.layers.Conv2D` inherit from `tf.keras.layers.Layer`, which in turn extends `tf.keras.Model`'s base functionality. The core polymorphic method `call()` enables subclass specialization, but execution is typically constrained by TensorFlow's computational graph tracing.

Thus, both frameworks achieve code reuse through inheritance and runtime polymorphism, yet PyTorch emphasizes flexibility and imperative control, while TensorFlow enforces encapsulation and graph-oriented consistency.

**Activation Function Coupling** TensorFlow couples activations tightly within layer definitions, as seen in `Dense(128, activation='relu')`. This design favors encapsulation and simplicity but violates the Single Responsibility Principle: a single class performs both linear transformation and non-linear activation. It also reduces extensibility, since changing the activation requires reinstantiating the layer. PyTorch, on the contrary, treats activations as separate components (`F.relu()`, `F.softmax()`), promoting loose coupling and composition. Each module has a single responsibility that enables flexible reordering, conditional execution, or custom activation insertion. Thus, TensorFlow prioritizes abstraction, while PyTorch adheres more closely to composition and the Open/Closed Principle.

## Optimization Design

**TensorFlow Optimization** In TensorFlow, model optimization is managed declaratively through the `compile()` and `fit()` interfaces:

```
model.compile(optimizer='adam', loss='
    sparse_categorical_crossentropy')
model.fit(x_train, y_train, epochs=3)
```

Listing 1: TensorFlow optimization example

Here, the optimizer and the loss function are configured at a high level, while the `fit` // method internally performs all steps of the training algorithm: forward propagation, loss computation, gradient calculation, parameter updates, and metric recording. The user does not directly manipulate any of these stages.

This design reflects two core object-oriented patterns:

- **Template Method Pattern:** The `fit()` method defines the *skeleton* of the training algorithm, while subclasses override `call()` to specify the forward pass. This enforces a consistent workflow while allowing model-specific customization.
- **Facade Pattern:** The `Model` class acts as a high-level facade, hiding multiple interacting subsystems—optimizers, losses, metrics, callbacks, and data pipelines—behind a single, simplified interface (`fit()`). This abstraction improves usability and encapsulation,

enabling users to train complex models with a single method call.

By combining these two patterns, TensorFlow achieves a strongly encapsulated, framework-driven optimization process: users define *what* to optimize, while the framework determines *how* optimization is executed.

**PyTorch Optimization** PyTorch exposes optimization as a user-managed process that favors flexibility and explicit control:

```
optimizer = torch.optim.Adam(model.
    parameters())

for x, y in dataloader:
    optimizer.zero_grad()
    output = model(x)
    loss = criterion(output, y)
    loss.backward()
    optimizer.step()
```

Listing 2: PyTorch optimization example

Here, the user implements the entire training loop: forward pass, loss evaluation, gradient computation, and parameter updates. This design exemplifies **composition over inheritance**—the optimizer and loss function are composed into the loop as independent, interchangeable strategy objects. It offers greater flexibility and transparency at the cost of additional user responsibility.

**Single Responsibility Principle** TensorFlow violates the Single Responsibility Principle in its optimization design: the `Model` class not only defines the network architecture but also manages the optimizer, loss computation, training loop, gradient updates, and callbacks through `fit()`. This centralization couples multiple responsibilities into a single class.

In contrast, PyTorch follows SRP by separating concerns clearly. The `Module` class handles only the forward computation, the `Optimizer` updates parameters, and the training loop orchestrates these components explicitly. Each object has a single, well-defined purpose, making the design more modular, testable, and extendable.

## Dataset and DataLoader Design

TensorFlow's `tf.data.Dataset` follows a declarative and encapsulated design. It combines multiple responsibilities—data access, batching, and parallelization—within a single class, exposing a fluent interface through method chaining (`.map()`, `.batch()`, `.prefetch()`). This design is an example of the *Builder* and *Template Method* patterns: users construct data pipelines step by step by chaining methods, while the framework encapsulates the internal logic for execution. The builder-style interface provides high abstraction and ease of use, but it also introduces tight coupling between the data pipeline and TensorFlow's execution graph, limiting extensibility and user control.

In contrast, PyTorch adopts an explicit, compositional architecture built on the *Strategy* and *Iterator* patterns. The `Dataset`, `Sampler`, and `DataLoader` classes act as

independent yet cooperating components that adhere to the Single Responsibility and Open/Closed Principles. The `Dataset` defines how samples are accessed, the `Sampler` determines the order in which samples are drawn, and the `DataLoader` manages batching, shuffling, and parallel loading. These components interact through the Iterator pattern, exposing a uniform interface (for `batch` in `DataLoader: ...`) that abstracts away the underlying data sources. This modular structure enables high extensibility—users can easily replace or extend any strategy independently—while maintaining a consistent data access interface. The trade-off is that users must explicitly compose and manage these components, gaining flexibility at the cost of higher control overhead.

```
from torch.utils.data import Dataset,
    DataLoader
import torch

# Custom dataset
class MyDataset(Dataset):
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels
    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]
    def __len__(self):
        return len(self.data)

# Composition: Dataset + DataLoader
dataset = MyDataset(torch.randn(1000, 32),
    torch.randint(0, 10, (1000,)))
loader = DataLoader(dataset, batch_size=32,
    shuffle=True)

# Iteration via Iterator pattern
for batch_x, batch_y in loader:
    print(batch_x.shape, batch_y.shape)
```

Listing 3: PyTorch dataset and dataloader example

Here, `DataLoader` acts as the context that orchestrates data access through interchangeable sampling and batching strategies, while the `Dataset` and `Sampler` define independent behaviors. This composition-based design exemplifies the Strategy pattern and promotes modular reusability—contrasting with TensorFlow’s tightly encapsulated, builder-style data pipeline.

## Conclusion

PyTorch and TensorFlow both implement core object-oriented ideas but embody distinct philosophies. PyTorch’s design promotes clarity and flexibility through explicit control of training and data flow, aligning with composition and single-responsibility principles. TensorFlow, in contrast, abstracts complexity into unified interfaces that enhance usability but reduce transparency. Understanding these trade-offs illustrates how software design patterns shape the balance between control and abstraction in modern deep learning frameworks.

## References

- [Abadi et al. 2016] Abadi, M.; Barham, P.; Chen, J.; Chen, Z.; Davis, A.; Dean, J.; Devin, M.; Ghemawat, S.; Irving, G.; Isard, M.; et al. 2016. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. *arXiv preprint arXiv:1603.04467*.
- [Andersen and Maalej 2023] Andersen, J. S., and Maalej, W. 2023. Design patterns for machine learning based systems with human-in-the-loop. *IEEE Software*.
- [Brown, Suryadinata, and others 2023] Brown, V.; Suryadinata, S.; et al. 2023. *Machine Learning Design Patterns*. O’Reilly Media.
- [Chen et al. 2024] Chen, K.; Bi, Z.; Wang, T.; Wen, Y.; et al. 2024. Deep learning and machine learning: Advancing big data analytics and management with design patterns. *arXiv preprint arXiv:2409.19916*.
- [Gamma et al. 1994] Gamma, E.; Helm, R.; Johnson, R.; and Vlissides, J. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [Keras official documents 2023] Keras official documents. 2023. Keras: The high-level api for tensorflow. <https://www.tensorflow.org/guide/keras>. Accessed: 2025-11-08.
- [Paszke et al. 2019] Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J.; Chanan, G.; Killeen, T.; Lin, Z.; Gimelshein, N.; Antiga, L.; Desmaison, A.; Kopf, A.; Yang, E.; DeVito, Z.; Raison, M.; Tejani, A.; Chilamkurthy, S.; Steiner, B.; Fang, L.; Bai, J.; and Chintala, S. 2019. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems* 32.
- [PyTorch official documents 2024] PyTorch official documents. 2024. Pytorch documentation. <https://pytorch.org/docs/stable/>. Accessed: 2025-11-08.
- [Wikipedia Pytorch 2025] Wikipedia Pytorch. 2025. PyTorch — Wikipedia. <https://en.wikipedia.org/wiki/PyTorch>. Accessed: 2025-11-08.
- [Wikipedia Tensorflow 2025] Wikipedia Tensorflow. 2025. TensorFlow — Wikipedia. <https://en.wikipedia.org/wiki/TensorFlow>. Accessed: 2025-11-08.